

Lab 08 Collections (*SUBMIT ALL YOUR CODE ON MOODLE*)

1. [10 points] Collection manipulations, Contact Manager App

- a) Define the Printable Interface, which has one method called printAll():

```
public void printAll();
```

- b) Create a Contact class that implements Comparable has:

- id:String
- name :String
- mobileNumber : String
- email : String

The class should also implement Comparable Interface. Two contacts are compared based on their name field.

Implement a no-argument Constructors and another one that initializes all attributes.

Implement getters and setters. Override the toString() and equals() methods .

- c) Define an abstract class called ContactManager that has the following abstract methods.

```
public abstract void addContact(String id, String name, String mobile, String email);  
public abstract void addContact(Contact contact);  
public abstract Contact getContactByEmail(String email);  
public abstract Collection<Contact> getContactsByAreaCode(String areaCode);
```

- d) Define a class called ContactManagerLinkedList that extends ContactManager and implements Printable. It should have a field:
contactsList: which is a LinkedList that holds Contact objects.

Implement a no-argument Constructors and one that initializes all attributes. Implement getters, setters and all other methods.

- e) Define a class called ContactManagerStack that extends ContactManager and implements Printable. It should have a field:
contactsList: which is a Stack that holds Contact objects.

Implement a no-argument Constructors and one that initializes all attributes. Implement getters, setters and all other methods. Use the .push() and .pop() methods of the stack.

- f) Define a class called `ContactManagerHashMap` that extends `ContactManager` and implements `Printable`. It should have a field:
`contactsList`: which is a `HashMap` that holds `Contact` objects.

Implement a no-argument Constructors and one that initializes all attributes. Implement getters, setters and all other methods.

- g) Define a class called `ContactManagerTreeMap` that extends `ContactManager` and implements `Printable`. It should have a field:
`contactsList`: which is a `TreeMap` that holds `Contact` objects.

Implement a no-argument Constructors and one that initializes all attributes. Implement getters, setters and all other methods.

- h) Define a static method in the main class called :

```
public static int loadContactsFromFile( String fileName, ContactManager contactManager )
```

Which opens the file “src\ \contacts.csv”, reads the contacts from there, save them inside the `contactManager` object (parameter), and close the file.

- i) Define the `main()` method that programs the problems:

Problem 1) create a `ContactManagerLinkedList` object, call `loadContactsFromFile` on that object, and print the contacts saved in the object using `praintAll()`.

Problem 2) create a `ContactManagerHashMap` object, call `loadContactsFromFile` on that object, and print the contacts saved in the object using `praintAll()`.

Problem 3) create a `ContactManagerTreeMap` object, call `loadContactsFromFile` on that object, and print the contacts saved in the object using `praintAll()`.

- j) Make sure to throw exceptions when needed, and handle them in the main code.
k) Whenever possible, traverse the Collection forward using iterator method.
l) Make sure the output looks like the sample output below. Use `String.format` in the `toString` methods.

Contact Manager App

Problem 1: using a Linked List with name as a key:
LinkedList Content:

001	Alejandro Bernal Alonso	5143065939	a.bernalalonso@gmail.com
002	Bianca Bien-Aimé	5148482107	b.bien-aimé@gmail.com
003	Olivier Dufort	4386559767	o.dufort@gmail.com
004	Victor Ivascu	4383479879	v.ivascu@gmail.com
005	Logan Kairns	4386308442	l.kairns@gmail.com
006	Ava Keyhani	5145488325	a.keyhani@gmail.com
007	Faizaan Khalid	4388313026	f.khalid@gmail.com
008	Zié Abdoul Malick Konate	4384574463	z.konate@gmail.com
009	Eric Kossovsky	4381706737	e.kossovsky@gmail.com
010	Mattys Leduc	4387374032	m.leduc@gmail.com
011	Cunningham Li	4386151345	c.li@gmail.com
012	Anthony Métras	4386700926	a.métras@gmail.com
013	Faiyazur Rahman Mir	4500160813	f.mir@gmail.com
014	Nima Miri	4385708124	n.miri@gmail.com
015	Jeremy Misola-Rellin	4382099601	j.misola-rellin@gmail.com
016	Jean-David Pallares	4506537466	j.pallares@gmail.com
017	Nathan Roos	4388231651	n.roos@gmail.com
018	Julian David Uribe Carrillo	4380121106	j.uribecarrillo@gmail.com
019	José Villegas Morilla	4388513844	j.villegasmorilla@gmail.com
020	Ryan Wade	4385400674	r.wade@gmail.com
021	Liam Wellard	4501542538	l.wellard@gmail.com

Problem 2: using a HashMap with name as a key:
Hash Map Content:

Key= 020	Value= " 020	Ryan Wade	4385400674	r.wade@gmail.com"
Key= 010	Value= " 010	Mattys Leduc	4387374032	m.leduc@gmail.com"
Key= 021	Value= " 021	Liam Wellard	4501542538	l.wellard@gmail.com"
Key= 011	Value= " 011	Cunningham Li	4386151345	c.li@gmail.com"
Key= 001	Value= " 001	Alejandro Bernal Alonso	5143065939	a.bernalalonso@gmail.com"
Key= 012	Value= " 012	Anthony Métras	4386700926	a.métras@gmail.com"
Key= 002	Value= " 002	Bianca Bien-Aimé	5148482107	b.bien-aimé@gmail.com"
Key= 013	Value= " 013	Faiyazur Rahman Mir	4500160813	f.mir@gmail.com"
Key= 003	Value= " 003	Olivier Dufort	4386559767	o.dufort@gmail.com"
Key= 014	Value= " 014	Nima Miri	4385708124	n.miri@gmail.com"
Key= 004	Value= " 004	Victor Ivascu	4383479879	v.ivascu@gmail.com"
Key= 015	Value= " 015	Jeremy Misola-Rellin	4382099601	j.misola-rellin@gmail.com"
Key= 005	Value= " 005	Logan Kairns	4386308442	l.kairns@gmail.com"
Key= 016	Value= " 016	Jean-David Pallares	4506537466	j.pallares@gmail.com"
Key= 006	Value= " 006	Ava Keyhani	5145488325	a.keyhani@gmail.com"
Key= 017	Value= " 017	Nathan Roos	4388231651	n.roos@gmail.com"
Key= 007	Value= " 007	Faizaan Khalid	4388313026	f.khalid@gmail.com"
Key= 018	Value= " 018	Julian David Uribe Carrillo	4380121106	j.uribecarrillo@gmail.com"
Key= 008	Value= " 008	Zié Abdoul Malick Konate	4384574463	z.konate@gmail.com"
Key= 019	Value= " 019	José Villegas Morilla	4388513844	j.villegasmorilla@gmail.com"
Key= 009	Value= " 009	Eric Kossovsky	4381706737	e.kossovsky@gmail.com"

Problem 3: using a TreeMap with name as a key:
TreeMap Content:

Key= 001	Value= 001	Alejandro Bernal Alonso	5143065939	a.bernalalonso@gmail.com
Key= 002	Value= 002	Bianca Bien-Aimé	5148482107	b.bien-aimé@gmail.com
Key= 003	Value= 003	Olivier Dufort	4386559767	o.dufort@gmail.com
Key= 004	Value= 004	Victor Ivascu	4383479879	v.ivascu@gmail.com
Key= 005	Value= 005	Logan Kairns	4386308442	l.kairns@gmail.com
Key= 006	Value= 006	Ava Keyhani	5145488325	a.keyhani@gmail.com
Key= 007	Value= 007	Faizaan Khalid	4388313026	f.khalid@gmail.com
Key= 008	Value= 008	Zié Abdoul Malick Konate	4384574463	z.konate@gmail.com
Key= 009	Value= 009	Eric Kossovsky	4381706737	e.kossovsky@gmail.com
Key= 010	Value= 010	Mattys Leduc	4387374032	m.leduc@gmail.com
Key= 011	Value= 011	Cunningham Li	4386151345	c.li@gmail.com
Key= 012	Value= 012	Anthony Métras	4386700926	a.métras@gmail.com
Key= 013	Value= 013	Faiyazur Rahman Mir	4500160813	f.mir@gmail.com
Key= 014	Value= 014	Nima Miri	4385708124	n.miri@gmail.com
Key= 015	Value= 015	Jeremy Misola-Rellin	4382099601	j.misola-rellin@gmail.com
Key= 016	Value= 016	Jean-David Pallares	4506537466	j.pallares@gmail.com
Key= 017	Value= 017	Nathan Roos	4388231651	n.roos@gmail.com
Key= 018	Value= 018	Julian David Uribe Carrillo	4380121106	j.uribecarrillo@gmail.com
Key= 019	Value= 019	José Villegas Morilla	4388513844	j.villegasmorilla@gmail.com
Key= 020	Value= 020	Ryan Wade	4385400674	r.wade@gmail.com
Key= 021	Value= 021	Liam Wellard	4501542538	l.wellard@gmail.com

Process finished with exit code 0

2. [15 points] Stack Practice Exercise: Text Editor with Undo/Redo Functionality

Problem Description

Create a text editor simulation that uses **two Stacks** to implement undo and redo functionality. Students will learn Stack operations while building a practical application.

Requirements

Part A: Create the Command Interface [2 points]

Define an interface called Command with the following methods:

```
public interface Command {  
    void execute();  
    void undo();  
    String getDescription();  
}
```

Part B: Create Concrete Command Classes [4 points]

Implement the following command classes that implement the Command interface:

1. InsertTextCommand

- **Fields:**
 - document: StringBuilder (reference to the document)
 - text: String (text to insert)
 - position: int (where to insert)
- **execute():** Insert text at the specified position
- **undo():** Remove the inserted text
- **getDescription():** Return description like "Insert 'Hello' at position 0"

2. DeleteTextCommand

- **Fields:**
 - document: StringBuilder
 - deletedText: String (stores deleted text for undo)
 - startPosition: int
 - length: int
- **execute():** Delete text from startPosition for specified length
- **undo():** Re-insert the deleted text
- **getDescription():** Return description like "Delete 5 characters at position 10"

3. ReplaceTextCommand

- **Fields:**
 - document: StringBuilder
 - oldText: String
 - newText: String
 - position: int
- **execute():** Replace old text with new text
- **undo():** Restore the old text
- **getDescription():** Return description like "Replace 'cat' with 'dog' at position 5"

Part C: Create the TextEditor Class [5 points]

Create a TextEditor class with:

Fields:

- document: StringBuilder (the actual text content)
- undoStack: Stack (stores executed commands)
- redoStack: Stack (stores undone commands)

Methods:

```
public void executeCommand(Command cmd)
public void undo()
public void redo()
public void showDocument()
public void showHistory()
public String getDocumentContent()
```

Behavior:

- executeCommand(): Execute the command, push to undoStack, clear redoStack
- undo(): Pop from undoStack, call undo(), push to redoStack
- redo(): Pop from redoStack, call execute(), push to undoStack
- showHistory(): Display both stacks without modifying them

Part D: Create the Main Program [4 points]

Write a main() method that demonstrates:

1. Create a TextEditor instance
2. Execute a series of commands:
 - Insert "Hello World" at position 0
 - Insert " Java" at position 11
 - Delete 6 characters at position 6 (delete "World ")
 - Insert "Stack" at position 6
 - Replace "Stack" with "Stacks" at position 6
3. Show document after each operation
4. Perform 2 undo operations
5. Perform 1 redo operation
6. Show final document and history

Additional Requirements

- Handle edge cases (undo on empty stack, invalid positions)
- Throw appropriate exceptions with meaningful messages
- Use proper Stack methods: push(), pop(), peek(), isEmpty()
- Format output clearly with separators
- Include comments explaining Stack operations

Sample Output

=== Text Editor with Undo/Redo ===

Operation 1: Insert 'Hello World' at position 0
Document: [Hello World]

Operation 2: Insert ' Java' at position 11
Document: [Hello World Java]

Operation 3: Delete 6 characters at position 6
Document: [Hello Java]

Operation 4: Insert 'Stack' at position 6
Document: [Hello Stack Java]

Operation 5: Replace 'Stack' with 'Stacks' at position 6
Document: [Hello Stacks Java]

--- Performing UNDO ---

Undoing: Replace 'Stack' with 'Stacks' at position 6
Document: [Hello Stack Java]

--- Performing UNDO ---

Undoing: Insert 'Stack' at position 6
Document: [Hello Java]

--- Performing REDO ---

Redoing: Insert 'Stack' at position 6
Document: [Hello Stack Java]

=== Command History ===

Undo Stack (top to bottom):

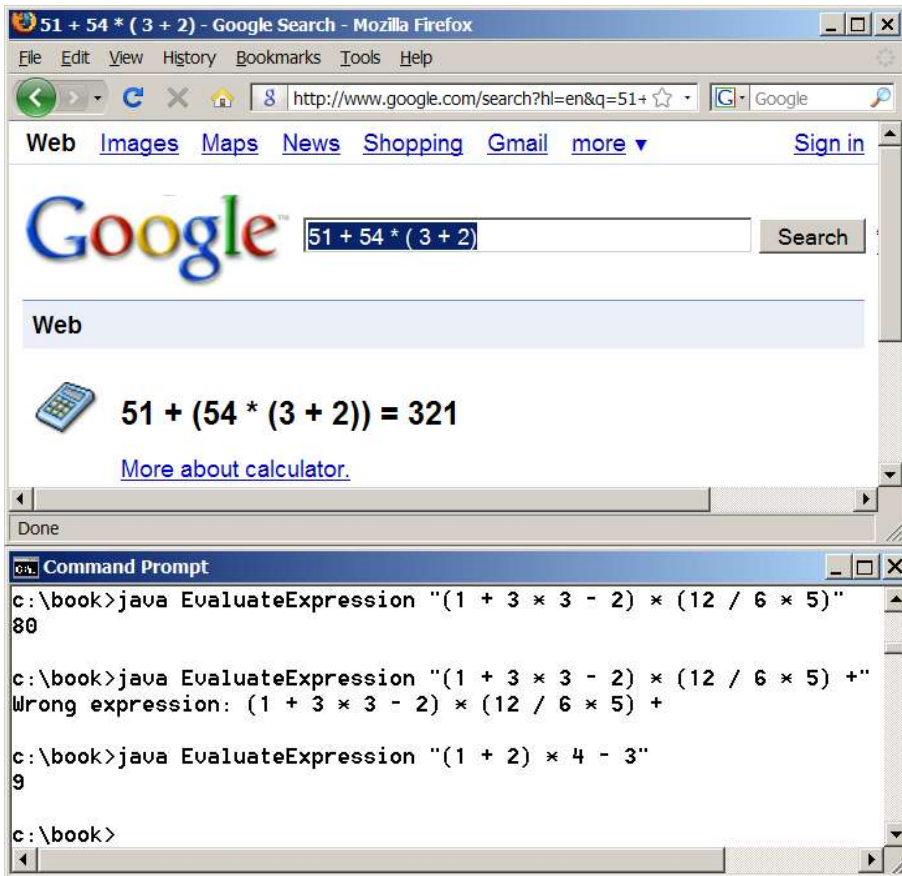
3. Insert 'Stack' at position 6
2. Delete 6 characters at position 6
1. Insert ' Java' at position 11
0. Insert 'Hello World' at position 0

Redo Stack (top to bottom):

0. Replace 'Stack' with 'Stacks' at position 6

Final Document: [Hello Stack Java]

3. BONUS [10 points] Case Study: Evaluating Expressions



- Stacks can be used to evaluate expressions.

Algorithm:

Phase 1: Scanning the expression

The program scans the expression from left to right to extract operands, operators, and the parentheses.

- 1.1. If the extracted item is an operand, push it to **operandStack**.
- 1.2. If the extracted item is a **+** or **-** operator, process all the operators at the top of **operatorStack** and push the extracted operator to **operatorStack**.
- 1.3. If the extracted item is a ***** or **/** operator, process the ***** or **/** operators at the top of **operatorStack** and push the extracted operator to **operatorStack**.
- 1.4. If the extracted item is a **(** symbol, push it to **operatorStack**.
- 1.5. If the extracted item is a **)** symbol, repeatedly process the operators from the top of **operatorStack** until seeing the **(** symbol on the stack.

Phase 2: Clearing the stack

Repeatedly process the operators from the top of **operatorStack** until **operatorStack** is empty.

Expression	Scan	Action	operandStack	operatorStack
(1 + 2)*4 - 3 ↑	(	Phase 1.4		(
(1 + 2)*4 - 3 ↑	1	Phase 1.1	1	(
(1 + 2)*4 - 3 ↑	+	Phase 1.2	1	+ (
(1 + 2)*4 - 3 ↑	2	Phase 1.1	2 1	(
(1 + 2)*4 - 3 ↑	)	Phase 1.5	3	
(1 + 2)*4 - 3 ↑	*	Phase 1.3	3	*
(1 + 2)*4 - 3 ↑	4	Phase 1.1	4 3	*
(1 + 2)*4 - 3 ↑	-	Phase 1.2	12	-
(1 + 2)*4 - 3 ↑	3	Phase 1.1	3 12	-
(1 + 2)*4 - 3 ↑	none	Phase 2	9	

- a- Write a class names ExpressionEvaluator that has the attributes expression of type String
operatorStack of type Stack<Character>
operandStack of type Stack<Integer>

Write its constructors, getters, and setters.

- b- Write a method in the class called : **public static String insertBlanks(String s)**, which makes sure that there is space between all operands and operators in the expression. test it.

Write the helper methods and test them:

public boolean isOperand(char c) : checks if c is a digit so operand

public boolean isOperator(char c) : checks if c is an operator

public void processAnOperator() : performs one operation

public void processAllOperatorsOnStack() : process all operations on the operator stack

processUntilParenthesis() : process all operations on the operator stack until you hit a '('

public void processAllMultDivsOnStack() : process all multiplication and division operations on the operator stack

- c- Write a method called public **int Phase1()** which implements the algorithm for phase 1. Test it.
- d- Write a method called public **int Phase1()** which implements the algorithm for phase 2. Test it.

